Data Cleaning & Preprocessing Report: Python Documentation

This document outlines the systematic data cleaning and feature engineering process performed using Python (Pandas) to prepare the raw retail datasets for SQL ingestion and Power BI visualization.

1. Library Initialization & Data Ingestion

The cleaning process began by importing the necessary Python libraries for data manipulation and visualization.

- **Libraries used:** pandas for data handling, matplotlib for initial EDA, and for statistical visualization.
- **Action:** Five primary datasets (customers.csv, products.csv, salesdata.csv, stores.csv, and returns.csv) were loaded into individual Data Frames.

2. Customers Data Cleaning

- **Deduplication:** Identified and removed duplicate rows to ensure a unique customer_id for every record.
- **Missing Value Imputation:**
  - Found missing values in the age column.
  - **Logic:** Calculated the median age specifically for "Male", "Female", and "Other" gender groups and filled the respective nulls. This ensured age distribution remained representative of each demographic.
- **Data Type Correction:** Converted signup_date from a string object to a datetime format and cast age to an integer.
- **Feature Engineering:** Created the age_group column using numerical binning to categorize customers into segments (e.g., 18-30, 31-45, 46-60, 60+).
- **Standardization:** Stripped whitespace and standardized text to "Title Case" for the region and gender columns.

3. Products Data Cleaning

- **Null Handling:** Identified nulls in the brand column and filled them with the string "Unknown" to maintain data integrity for brand-level analysis.
- **Financial Validation:** Checked for negative values in unit_price and cost_price.
- **Feature Engineering:** * **Margin Percentage:** Developed a calculation for margin_pct using the formula: ((unit_price - cost_price) / unit_price).
    - This metric was critical for identifying "Profit Killers" in the later SQL analysis.
- **Standardization:** Applied uniform casing to the category and product_name fields.

4. Sales Data Cleaning

- **Relational Merging:** To perform profit calculations, the Sales table was temporarily merged with the Products table on product_id to access cost information.
- **Handling Leap Year Anomalies:** Identified and corrected "out-of-range" dates (e.g., Feb 29th on non-leap years) to ensure the order_date could be converted to a proper datetime object.
- **Missing Values:** Filled missing store_id values with "Online" to accurately represent digital sales channels.
- **Calculated Columns:**
    - **Total Amount:** Calculated as (unit_price * quantity) * (1 - discount_pct).
    - **Net Profit:** Derived by subtracting the total cost from the total sales amount.

5. Stores & Returns Data Cleaning

- **Stores:** Validated operating_cost as a numeric field and standardized geographic labels (City, Region) to match the Customers table for accurate cross-filtering.
- **Returns:** Standardized the return_reason column into four distinct categories: *Defective, Wrong Item, Late Delivery,* and *No Longer Needed*.

- **Date Synchronization:** Ensured return_date was formatted identically to order_date to allow for "Time-to-Return" analysis.

6. Exploratory Data Analysis (EDA) & Export

Before finalizing the cleaning phase, a final validation check was performed:

- **Visual Validation:** Created a bar chart showing **Total Revenue by Product Category** to ensure the cleaned sales data aligned with expected financial totals.
- **Data Export:** The cleaned DataFrames were exported to .csv format (e.g., customers_cleaned.csv, salesdata_cleaned.csv) for seamless ingestion into **SQL Server**.

Author: Sara Mahesh

Project: End-to-End Retail Performance and Behavioral Analytics

Tools: Python (Pandas, Matplotlib)